

MODULE 1: DATABASE DESIGN AND IMPLEMENTATION

UNIT 1: RELATIONAL DATABASE DESIGN AND IMPLEMENTATION

1. INTRODUCTION

This unit discusses the relational data model. Which uses the concept of mathematical relation, which looks somewhat like a table of values, as its basic building blocks, and has its theoretical basics in *set theory* and *first order predicate logic*. In this unit, we will discuss the basic characteristics for the relational model and its normalization processes. The model has been implemented in a large number of commercial systems over the last years.

1.1 OBJECTIVES.

The objectives of this unit include to:

1. Describe the data model and its concepts.
2. discuss the relational database model.
3. Help the students understand the concepts of normalization in database design

1.2 What is a data model?

A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. In this part, we focus on the relational model. It is a notation for describing data or information. The description generally consists of three parts: Structure of the data, Operations on the data and Constraints on the data. In unit we describe the relational database model

1.3 What Is Relational Database?

A relational database is based on the relational model and uses a collection of tables to represent both data and the relationships among those data. It also includes a Data Manipulation Language (DML) and Data Definition Language (DDL). The relational model is today the primary data model for commercial data processing applications - especially for storing financial records, manufacturing and logistical

information, personnel data and much more. It attained its primary position because of its simplicity, which eases the job of the programmer, compared to earlier data models such as the network model or the hierarchical model.

1.2.1 Structure of Relational Databases

A relational database consists of a collection of **tables**, each of which is assigned a unique name. For example, consider the *Facilitator* table (Table 2.1), which stores information about facilitators. The table has four columns: *ID*, *name*, *department* and *rank*. Each row of this table records information about a facilitator, consisting of the facilitator's *ID*, *name*, *department*, and *rank*. Similarly, the *course* table (Table 2.2) stores information about courses, consisting of a *course code*, *title*, *department*, and *Credit Hour(CH)*, for each course. Note that each instructor is identified by the value of the column *ID*, while each course is identified by the value of the column *course code*.

Table 2.1: Facilitator table

ID	Name	Department	Rank
001010	Patience	Computer Science	Reader
001023	Inyang	Chemistry	Professor
001058	Godwin	Physics	Lecturer 1
002010	Daniel	History	Professor
003010	Suleman	Geography	Reader

Table 2.2: Facilitator table

Course Code	Title	Department	CH
CIT 751	Patience	Computer Science	3
CHM 222	Inyang	Chemistry	4
PHY 234	Godwin	Physics	3
HIS 442	Daniel	History	2
GPY 130	Suleman	Geography	4

In general, a row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of *relation*, from which the relational data model takes its name. In mathematical terminology, a *tuple* is simply a sequence (or list) of values. A relationship between n values is represented mathematically by an n -tuple of values, i.e., a tuple with n values, which corresponds to a row in a table.

Thus, in the relational model the term relation is used to refer to a table, while term tuple is used to refer to a row. Similarly, the term attribute refers to a column header of a table while the data type describing the types of values that can appear in each column is called a domain. Examining Table 2.1, we can see that the relation instructor has four attributes: ID, name, department, and rank. We use the term **relation instance** to refer to a specific instance of a relation that is, containing a specific set of rows. The instance of a *facilitator* shown in Table 2.1 has 5 tuples, corresponding to five facilitators.

Each attribute of a relation, has a set of permitted values, called the **domain** of that attribute. Thus, the domain of the *rank* attribute of relation is the set of all possible ranks values, while the domain of the *name* attribute is the set of all possible *facilitators*' names. A domain is **atomic** if elements of the domain are considered to be indivisible units. For example, suppose the table *facilitator* had an attribute *phone number*, which can store a set of phone numbers associated to the *facilitator*. Then the domain of *phone number* would not be atomic, since an element of the domain is a set of phone numbers (may be more than one phone numbers), and it has subparts, namely each individual phone numbers in the set.

The important issue is not what the domain itself is, but rather how we use domain elements in our database. Suppose now that the *phone number* attribute stores a single phone number. Even then, if we split the value from the phone number attribute (for example +234-873-424-1626), into a country code, network provider, an area code and a local number, then it is considered as a non-atomic value. If view each phone number as a single indivisible unit, then the attribute *phone number* would then have an atomic domain.

1.2.2 Database Schema

A **database schema**, is the logical design of the database, and the **database instance**, is a snapshot of the data in the database at a given instant in time. A database schema represents the logical configuration of all or part of a relational database. It can exist both as a visual representation and as a set of formulas (known as integrity constraints), that govern a database. These formulas are expressed in a DDL, such as SQL. As part of a data dictionary, a database schema indicates how the entities that make up the database relate to one another, including tables, views, stored procedures, and more. At the most basic level, a database schema indicates which tables or relations make up the database, as well as the attributes included on each table. Thus, the terms schema diagram and entity-relationship diagram are often interchangeable. The concept of a relation corresponds to the programming-language notion of a variable, while the concept of a **relation schema** corresponds to the programming-language notion of type definition.

A database designer creates a database schema mainly to help programmers whose software will interact with the database. The process of creating a database schema is called **data modeling**. There are two main kinds of database schema:

1. A logical database schema conveys the logical constraints that apply to the stored data. It may define integrity constraints, views, and tables.
2. A physical database schema lays out how data is stored physically on a storage system in terms of files and indices.

1.2.3 What is Database instance or database schema?

These terms, though related, do not mean the same thing. A database schema is a sketch of a planned database. It does not actually have any data in it. A database instance, on the other hand, is a snapshot of a database as it existed at a particular time. Thus, database instances can change over time, whereas a database schema is usually static, since it is arduous task to change the structure of a database once it is operational. Database schemas and database instances can affect one another through a database

management system (DBMS). The DBMS makes sure that every database instance complies with the constraints imposed by the database designers in the database schema.

1.2.3 Database Schema Integration

Schema integration describes the task of building a global data schema from a set of local schemas (usually with overlapping semantics) to provide the user with a unified view of the entire dataset. Schema integration is defined as the process of merging several conceptual schemas into a global conceptual schema that represents all the requirements of the application. Schema integration is used to merge two or more database schemas into a single schema that can store data from both the original databases.

The goal is to give the user of the global schema an illusion of a single dataset specified by a single schema.

1. Schema integration is used when two or more existing databases must be combined, for example, when a new management information system is being developed.
2. Schema integration may be used when the process of database design is too large to be carried out by one individual. Two or more designers will build models of different parts of the database and use schema integration to merge the resulting models.

There are two major types of schema integration:

1. **View Integration:** View integration takes place during the design of a new database when user requirements may be different for each user group. View integration is used to merge different viewpoints into a single data model.
2. **Database Integration:** Database integration is used when two or more databases must be combined to produce a single schema, called a global schema.

Reasons for Schema Integration

1. The structure of the database for large applications is too complex to be modeled by a single designer in a single view.
2. User groups typically operate independently in organization and have their own requirements and expectations of data, which may conflict with other user groups
3. When two database schemas are designed by different designers using different user requirements, the resulting schemas will often present contrasting views of the same data.

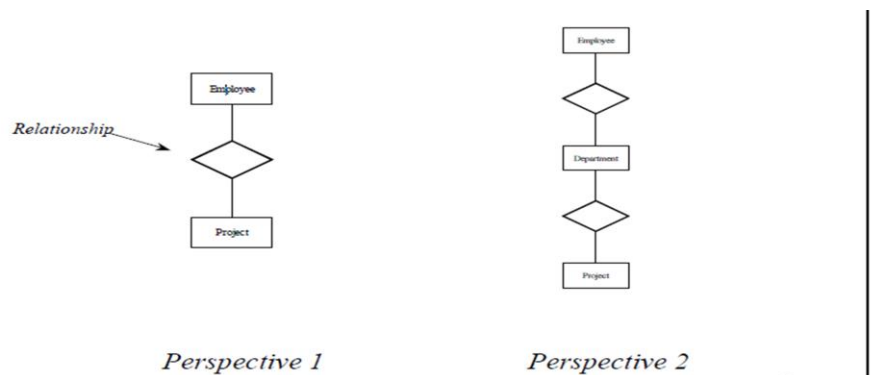


Figure 2.1: Two schemas designed differently

For example above, the relationship between employee and project in one database is represented as a relationship between employee, department and project in another database.

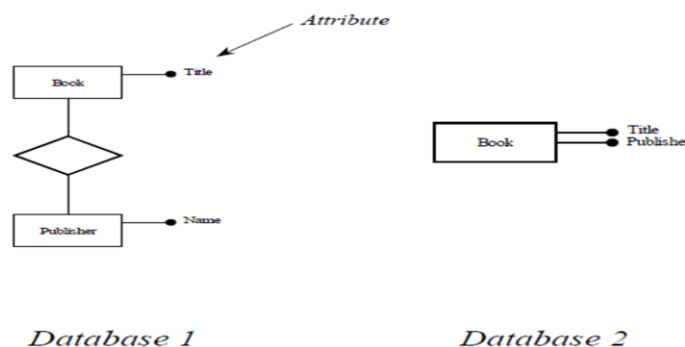


Figure 2.2: entity relationship in two databases

This situation might occur in an organization that allows different departments to have different rules as to how employees are allocated to projects. For example, in one department employees may be assigned to projects while in another department employees may not be considered to be directly related to a project.

Different databases may treat the same concepts in different ways.

In the above example, the publisher concept is an entity in database 1 but an attribute in database 2. There are two situations that must be dealt with during schema integration:

1. When different concepts are modelled in the same way. For example, in a university database staff and students may be represented by the entity person even though they are different concepts.
2. When the same concepts are modelled in different way. For instance, the above example models the concept of a publisher as an entity and as an attribute.

Incompatible designs:

1. Two database designs may be incompatible because mistakes were made in the initial design or there are different constraints placed on the data. For instance, in the above example, the relationship between employee and project is represented as a one-to-many relationship in database 1 and as a many-to-many relationship in database 2.
2. This problem may be caused by mistakes made during the initial database analysis task or because users of the system have different working practices. For example, one department in an organization, which works on small projects, may allocate one employee to a project but a different department, which works on large projects, may allocate many employees to a project.

During schema integration these different viewpoints must be reconciled

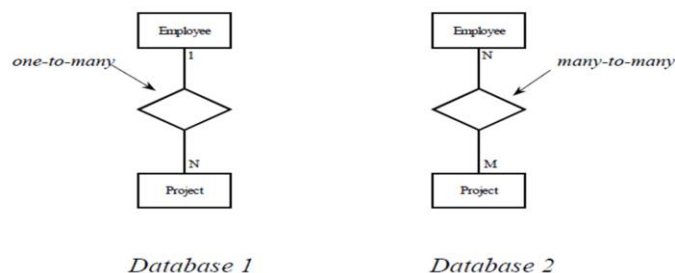


Figure 2.3: schemas reconciliation

Steps and goals of the integration process

i) Pre-integration.

1. Choose integration processing strategies
2. This governs the choice of schemas to be integrated

ii) Comparison of the schemas.

Schemas are analyzed and compared to determine the correspondences among concepts and detect possible conflicts.

iii) Conforming the schemas.

Once conflicts are detected, an effort is made to resolve them so that the merging of various schemas is possible.

Automatic conflict resolution is generally not feasible; interaction with designers is required.

iv) Merging and Restructuring.

1. The schemas are ready to be superimposed, giving rise to some intermediate integrated schema(s).
2. The second strategy for schema integration is to integrate some of the schemas (e.g. two) and then to integrate the resulting schemas.
3. This approach would be more appropriate when the schemas are complex or when there are a large number of schemas

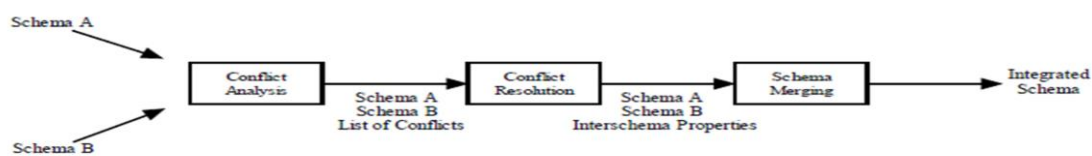


Figure 2.4: schema integration process

The schema integration process starts with two or more schemas and involves three main stages:

1. **Conflict Analysis:** during conflict analysis differences in the schemas are identified, for example, similar concepts that are represented in different ways. Check all conflicts in the representation of the same objects in different schemas.

2. **Conflict Resolution:** resolution the conflicts identified during conflict analysis are resolved. For example, a common method of representing equivalent concepts will be decided upon. This process may involve discussing the problems with the users or correcting errors in the schemas.
3. **Schema merging:** During schema merging the schemas are merged into a single schema using the decisions made during the conflict resolution.

Types of Conflicts

1. **Naming Conflicts:** It occurs when different names are used for the same attribute or the same object are given different names in their respective databases. Different naming conventions are used by databases for objects.
1. **Structural conflicts.** This type of conflict occurs when the actual method of representing the same concept in different databases is different or incompatible.

Types of Name conflicts :

1. **Synonyms:** Objects that are the same but have different names. For example, two public transport databases may have entities called passenger and customer, SNO and S#, EMP and EMPLOYEE, TAKE and ENROL. These entities may be the same entity.
2. **Homonyms:** Concepts or objects that are different but have the same names.. For example, two publishing databases may have entities called publication but in one database a publication may be a book while in the other database a publication may be a journal.
3. **Name conflicts** cause a problem because information may be duplicated in the integrated database. It is important to identify those data items in each schema that actually represents the same concept or that should be represented using different structures in the integrated schema.
4. **Synonyms** may be removed from the database by renaming the concepts so that they have the same name.
5. **Homonyms** may be removed from the database by renaming the concepts so that they have different names.

6. It may be possible to use a superclass/subclass relationship to avoid synonyms or homonyms.

Types of Structural Conflicts: There are three cases

1. **Identical Concepts:** When the same concept in different databases is represented in the same way they may be merged. For example, when an entity publication has the same structure and means the same in two databases the entities may be merged.
2. **Compatible Concepts:** When the same concept in different databases is represented in compatible ways they may be merged. For example, when an entity publication is represented by an attribute in one database and an entity in another database they may be merged by converting the attribute into an entity.
3. **Incompatible Concepts:** When the same concept in different databases is represented using different structures then it may be difficult to merge them directly. For example: Relationships may have different cardinalities (that is, one-to-many and many-to-many). This is also referred to as **cardinality conflicts** of attributes (that is different cardinalities for the same attributes of an entity types in more than one views).
4. Primary keys may be different, that is, Different keys are assigned as identifiers of the same entity type in different schemas.
5. Set relationships may be reversed (e.g. projects contain programmes and programmes contain projects).
6. Incompatible designs must be resolved by re-analyzing the data and adapting one or more of the schemas or by constructing a new, common representation.

Data type conflicts

The same attribute with different data types. For example, integer & real $\Rightarrow$ integer e.g. char (20) and char (30) $\Rightarrow$ char (20) e.g. different range values positive integer & [1,100] $\Rightarrow$ [1,100]

1.1.4 Database Design

In this section, the design issues regarding relational databases are described. In general, the goal of a relational database design is to generate a set of relation schemas that allow us to store information without unnecessary redundancy, yet allowing us to retrieve information easily. A well-structured and efficient database has the following advantages:

1. Saves disk space by eliminating redundant data.
2. Maintains data accuracy and integrity.
3. Provides access to the data in useful ways.

Designing an efficient, useful database is a matter of following the proper process, including these phases:

1. Strategy and planning
 2. Requirements analysis
 3. Design
 4. Development
 5. Deployment/implementation
 6. Operations and maintenance.
-
1. **Strategy and planning** – typically the cycle starts with the strategy and planning phase to identify the need and scope of a new system.
 2. **Specification Requirements analysis phase** – a more detailed requirements analysis will be carried out which will include identifying what the users require of the system; this will involve conceptual analysis.
 3. **Design phase** – this will involve producing a conceptual, logical and physical design. To undertake these processes it is important to be able to understand and apply the data modelling techniques which are covered in this book. When a suitable logical design has been obtained the development phase can begin.
 4. **Development phase** – this involves creating the database structure using an appropriate Database

5. **Management System** (DBMS) and usually includes the development of applications that provide a user interface consisting of forms and reports which will allow controlled access to the data held in the database.
6. **Deployment/implementation** – when the system has been developed it will be tested, it will then be deployed ready for use.
7. **Operations and maintenance** – following the system release for use it will be maintained until it reaches the end of its useful life, at this stage the development lifecycle may restart.

1.1.5 Specification Requirements Gathering

The most critical aspect of specification is the gathering and compilation of system and user requirements. This process is normally done in conjunction with managers and users. The initial phase of database design is to characterize fully the data needs of the prospective database users. The database designer needs to interact extensively with domain experts and users to carry out this task. The outcome of this phase is a specification of user requirements. While there are techniques for diagrammatically representing user requirements, in this unit we restrict ourselves to textual descriptions of user requirements

The major goals in requirements gathering is to:

- collect the data used by the organization,
- identify relationships in the data,
- identify future data needs, and
- determine how the data is used and generated.

The starting place for data collection is gathering existing forms and reviewing policies and systems. Then, ask users what the data means, and determine their daily processes.

These things are especially critical:

- Identification of unique fields (keys)
- Data dependencies, relationships, and constraints (high-level)
- The data sizes and their growth rates

Fact-finding is using interviews and questionnaires to collect facts about systems, requirements, and preferences. Five fact-finding techniques:

1. examining documentation (example invoices, invoices, timesheets, surveys etc.),
Comb through any existing data systems (including physical and digital files)
2. interviewing
3. observing the enterprise in operation
4. research
5. questionnaires
6. Start by gathering any existing data that will be included in the database. Then list the types of data you want to store and the entities, or people, things, locations, and events, that those data describe, for example:

1. Customers
2. Name
3. Address
4. City, State, Zip
5. Email address
6. Products
7. Name
8. Price
9. Quantity in stock
10. Quantity on order
11. Orders
12. Order ID
13. Sales representative
14. Date
15. Product(s)
16. Quantity
17. Price
18. Total

This information will later become part of the data dictionary, which outlines the tables and fields within the database. Be sure to break down the information into the smallest useful pieces. For instance, consider separating the street address from the country so that you can later filter individuals by their country of residence. Also, avoid placing the same data point in more than one table, which adds unnecessary complexity.

The result of this step is concisely written as a set of users' requirements. These requirements should be specified as detailed and complete a form as possible. In parallel with specifying the data requirements, it is useful to specify the known functional requirements of the application. These consist of the user-defined operations (or transactions) that will be applied to the database and they include both retrievals and updates. In software design, it is common to use data flow diagrams, sequence diagrams, scenarios, and other techniques for specifying functional requirements

1.1.6. Database Design Phase

The requirements gathering and specification provides you with a high-level understanding of the organization, its data, and the processes that you must model in the database. Database design involves constructing a suitable model of this information. Since the design process is complicated, especially for large databases, database design is divided into three phases:

- Conceptual database design
- Logical database design
- Physical database design

1. Conceptual Schema Modelling

Once all the requirements have been collected and analyzed, the next steps is to create a conceptual schema for the database, using a high-level conceptual data model. That is to develop layout or a visual representation of the proposed database.

Why do you need to a Conceptual model?

In many environments modelling is used to ensure that a product will satisfy the user's requirements before it is produced. For example, an architect may use a scale model of a building so the client can see what it will look like before it is built. This allows for any changes to be made to the design following feedback and before any expensive building

work takes place. Similarly, a modelling approach is needed when designing a database system so that interested parties can check that the design will satisfy the requirements.

How can a database system be modelled?

In order to design an effective database system you need to be able to understand an organization's information needs and, in particular, identify the data needed to satisfy these needs. Entity Relationship is an important top-down analysis technique which is used to show the structure of the data used by a system. Initially, a conceptual model is produced which is independent of any hardware or DBMS system; this is achieved by using an Entity Relationship Diagram (ERD) or alternatively a UML Class Diagram (CD). This modelling technique will be used to determine how this business data is structured and show the relationships between the different data entities. The model forms the basis for the design of the database system that will be built.

1.1.7 The Entity-Relationship Model

The entity-relationship (E-R) data model was developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database. The E-R model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the E-R model.

1.1.7.1 The Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) shows “entities” and the “relationships” that link them. The entities represent the data items needed by the system and the relationships show how the entities are related to one another. An “entity” is formally called an “entity type” and can be defined as: “A group of objects with the same properties which are identified by the enterprise as having an independent existence”

1.1.7.2 Entity types

In order to produce an ERD you need to identify all the entity types that are relevant to the system being modelled. Do not confuse an entity type with the occurrence of an

entity. Often many entities can be identified, although they are not always relevant to the needs of the system being considered, so care needs to be taken to ensure that only those that are needed are added to the ERD. The following are examples of typical entity types:

For a business system: CUSTOMER, ORDER, INVOICE.

For a university system: STUDENT, LECTURER, COURSE

Entities often fall into one of the following categories:

1. Physical – CAR, BUILDING
2. Human – CUSTOMER, EMPLOYEE
3. Place – FACTORY, SCHOOL
4. Group – DEPARTMENT, TEAM
5. Document – INVOICE, PAYSLIP

When you have identified the entity types, these need to be added to the Entity Relationship Diagram (ERD). Although ERDs can be drawn by hand, it is good practice to use a Computer Aided Software Engineering (CASE) tool to ensure your models can be amended easily and presented in a professional form to others. There are many CASE tools available to support modelling.

The E-R data model employs three basic concepts: entity sets, relationship sets, and attributes.

1.1.7.3 Entity Sets

An entity is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in a university is an entity. An entity has a set of properties, and the values for some set of properties may uniquely identify an entity. For instance, a person may have a person id property whose value uniquely identifies that person. Thus, the value 677-89-9011 for person ID would uniquely identify one particular person in the university. Similarly, courses can be thought of as entities, and course ID uniquely identifies a course entity in the university. An entity may be **concrete**, such as

a person or a book, or it may be abstract, such as a course, a course offering, or a flight reservation.

1. An **entity set** is a set of entities of the same type that share the same properties, or attributes. The set of all people who are instructors at a given university, for example, can be defined as the entity set instructor. Similarly, the entity set student might represent the set of all students in the university.
2. In the process of modeling, we often use the term entity set in the abstract, without referring to a particular set of individual entities. We use the term **extension of the entity set** to refer to the actual collection of entities belonging to the entity set. Thus, the set of actual instructors in the university forms the extension of the entity set instructor.
3. An entity is represented by a set of **attributes**. Attributes are descriptive properties possessed by each member of an entity set. The designation of an attribute for an entity set expresses that the database stores similar information concerning each entity in the entity set; however, each entity may have its own value for each attribute. Possible attributes of the *instructor* entity set are *ID*, *name*, *dept name*, and *salary*. In real life, there would be further attributes, such as street number, apartment number, state, postal code, and country, but we omit them to keep our examples simple. Possible attributes of the *course* entity set are *course ID*, *title*, *dept name*, and *credits*.
4. Each entity has a **value** for each of its attributes. For instance, a particular *instructor* entity may have the value 12121 for *ID*, the value for *name*, the value Finance for *dept name*, and the value 90000 for *salary*.
5. The ID attribute is used to identify instructors uniquely, since there may be more than one instructor with the same name. In the United States, many enterprises find it convenient to use the social-security number of a person² as an attribute whose value uniquely identifies the person. In general the enterprise would have to create and assign a unique identifier for each instructor.
6. A database for a university may include a number of other entity sets. For example, in addition to keeping track of instructors and students, the university

also has information about courses, which are represented by the entity set course.

1.1.7.4 *Entity selection and validation*

In order to produce the ERD you need to ensure you have identified the entities that are suitable for inclusion. The entities initially selected are usually referred to as **“candidate entities”** as not all may be suitable for inclusion. Entity names are normally **nouns** not verbs. The candidate entities are usually identified by referring to a written system description, a set of requirements, or perhaps the notes from a discussion with a person who has knowledge of the system under consideration.

These nouns will form the candidate entity list.

To ensure that a candidate entity is valid for inclusion on the ERD it should satisfy the following three checks:

1. It should not be the name of the system being modelled

It is a common mistake to include an entity which has the name of the system or organization that is being modelled. For example, if you were producing a model of “NOUN University” it would not be appropriate to include an entity type called NOUN UNIVERSITY or even UNIVERSITY as there is only one occurrence of this university. The whole model would, in reality, represent the university. However, if you were modelling a system that needed to hold data for more than one university, then you would need to include an entity type called UNIVERSITY.

2. The object should be of importance to the system being studied.

There are likely to be many objects in the system being studied but you have to decide whether the object is relevant. This usually means determining if the system users are likely to need to retrieve information about the object. For example, if you were designing a university student information system is a “litter bin” likely to satisfy the check? The answer would be no, but are there any circumstances in which it might? If the purpose of the system was to record all university assets, then

you might need to record information about the litter bins. In that case you would need an entity type to represent this information, though the entity type would be called ASSET and bin would be an entity occurrence.

3. There should be data attributes that can be associated with the entity

There must be at least two attributes for an entity type. If you cannot identify any or only one attribute for the entity then you may need to consider whether, in fact, it is actually an attribute of another entity type.

1.1.7.5 Validating the model

The model should be checked with the client or system users to ensure that all relevant entities have been identified, along with the required attributes. This process may need to be repeated a number of times until everyone is satisfied that all requirements have been met.

1.1.7.6 Entity Relationships

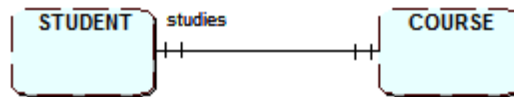
In order to see what is meant by a relationship, consider the following example which uses the music system entity Types COMPANY, CD, TRACK and CATEGORY.

There are a number of relationships between these entity types as follows:

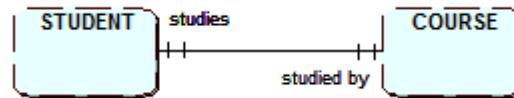
- A COMPANY produces CDs
- A CD contains music TRACKs
- A TRACK belongs to a music CATEGORY

To help you understand the nature of a relationship you may initially find it helpful to see the entity occurrences in a graphical format. If you consider the relationship “COMPANY produces a CD” the diagram below shows how one occurrence of COMPANY relates to two occurrences of CD when looked at from the viewpoint of the COMPANY, which is at the one end of the relationship. From this direction the relationship can be read as “A COMPANY produces CDs.”

If there is a relationship between an occurrence of one entity type and an occurrence of another entity type, then it is shown on the entity relationship diagram as a line linking the two entity symbols. The relationship between the two entities should be labelled by using a suitable verb. For example the relationship “STUDENT Studies a COURSE” would be represented as follows



As it is important to consider a relationship from both directions you should also label the relationship from COURSE to STUDENT as follows:



The relationship labels should be positioned as above, near to the relevant entities to aid readability.

Relationship cardinality

Once you have established a relationship between two entity types it is important to consider how many occurrences of one entity could be related to the other entity. This is referred to as “cardinality”.

There are three types of relationship cardinality:

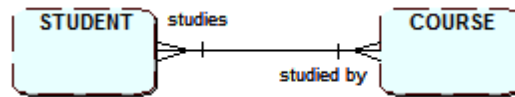
1. One to One (abbreviated as 1:1)
2. One to many (abbreviated as 1:M)
3. Many to Many (abbreviated as M:M or M:N)

Using the earlier example of the relationship between STUDENT and COURSE, consider the relationship from the STUDENT’s viewpoint. A student can study a course and if you then consider the relationship from the COURSE viewpoint, you can say that a COURSE can be studied by many STUDENTs. This would be a “**one to many**” (1:M) relationship and would be drawn on the ERD as follows:



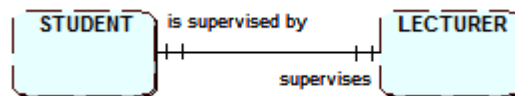
The “crow’s foot” symbol is used to represent many and is placed at the “many” end of the relationship. The relationship would be read formally as “a student studies one and only one course and a course is studied by one or many students”. If you now reconsider the relationship between STUDENT and COURSE but want to be able to show that a student may study more than one course, you now need to

alter the relationship to show as a “many to many” (M:M or M:N). A M:N relationship is sometimes written as M:M though M:N is preferred so as to indicate that the number of occurrences at one end of the relationship can be different from number at the other end of the relationship. This is drawn on the ERD as follows:



The final cardinality type that needs to be examined is for the “one to one” relationship. If a STUDENT is assigned a LECTURER as a supervisor and the LECTURER only supervises one student, you can show this as follows on the ERD:

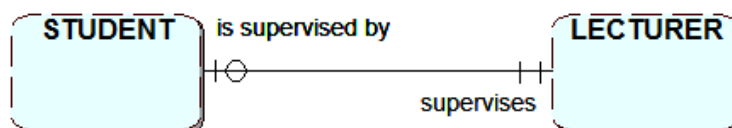
In this case the relationship would be read as “a student is supervised by one and only one lecturer and a lecturer supervises one and only one student



1.1.7.5 Relationship optionality

When describing an entity relationship you need to record the fact on the ERD that in some cases an occurrence of an entity type may not always be present, in which case the relationship is said to be **optional**. Using the previous cardinality example, the model states that a lecturer supervises a student. However, what if some lecturers do not act as supervisors to students? In this situation an occurrence of LECTURER will not always be related to an occurrence of STUDENT so it will be an optional relationship. However, if you consider the relationship from the STUDENT perspective it is still present as all students must have a supervising LECTURER.

To denote that a relationship can be optional a small circle is included on the relationship line at the end that is optional. The following shows the optional 1:1 relationship between STUDENT and LECTURER



There are various types of relationships

1. Complex Relationship
2. Recursive relationships

So far the concentration is on identifying and modelling relationships between pairs of entity types. Most of these relationships will be one-to-many, a few might be many-to-many and some might be one-to-one. In addition, how to resolve many-to-many relationships that contain data which is of interest in the situation being modelled has also been discovered.

Some entities are related to themselves. To be more specific, occurrences of the entity type are related to other occurrences of the same entity type. This is called a recursive relationship.

Consider the entity type EMPLOYEE in a university where there are approximately 500 employees, resulting in 500 occurrences of the entity. The Vice-chancellor manages the Deans of Faculty and each Dean manages several Heads of Department. The Heads of Department manage the lecturers. This gives rise to a hierarchical relationship within this single EMPLOYEE entity type. This can be represented graphically using a hierarchy diagram, as follows:

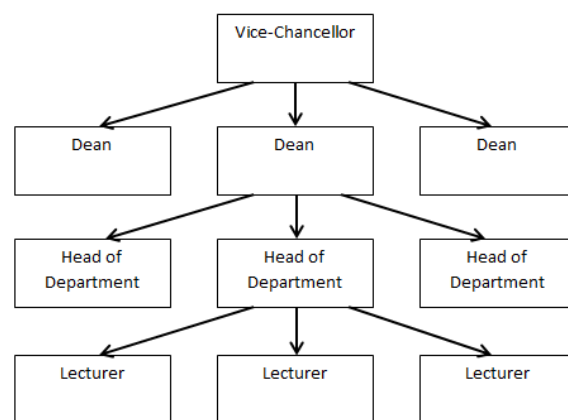
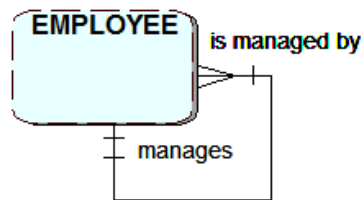


Figure 2.5: Illustration of hierarchical relationship of entity ‘EMPLOYEE’

This hierarchy diagram clearly shows that an occurrence of the entity EMPLOYEE, say Dean, manages one or more other occurrences of EMPLOYEE. Another occurrence of the entity, a Head of department, also manages one or more other

occurrences of the same entity, Lecturer. To show this 1:M recursive relationship on an ERD you draw a relationship line starting and finishing at the entity, as follows:



Exclusive relationships

Sometimes two or more relationships are mutually exclusive, e.g. a VEHICLE may be undergoing a SERVICE or an INSPECTION but not both at the same time. This is shown by an arc symbol pointing towards the mutually exclusive options.

1.1.8 Logical Design Phase

In the **logical-design** phase, the designer maps the high-level conceptual schema onto the implementation data model of the database system that will be used. The implementation data model is typically the relational data model, and this step typically consists of mapping the conceptual schema defined using the entity-relationship model into a relation schema.

It includes the following activities:

Identification of keys

Keys play a vital role in database design and have to be identified and used correctly. The following terminology is used in association with relational database keys:

- a key uniquely identifies an entity occurrence: it is the entity identifier
- a primary key is the key 'chosen' for a given relation / table
- a candidate key is a 'possible' primary key (several candidate keys may exist for a relation)
- a compound key is a key consisting of two or more attributes

1. Identification of relations

From your conceptual data model you need eventually to generate a set of relations (tables) that will form the basis of the database. You will need to link these tables in order to be able to reflect the relationships that were modelled on the ERD at the conceptual stage. In order to get to the stage of producing the tables, you first need to produce a complete set of relations in which all of the keys have been identified

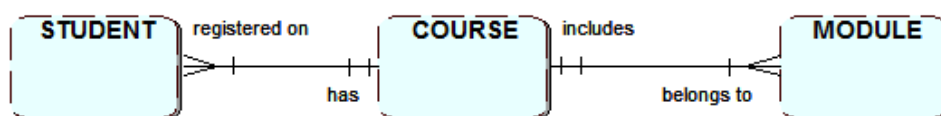
1. **Normalize the** database (Resolve relationships issues)
2. -- **Un-normalised (UNF)**,
3. -- 1st Normal Form (1NF),
4. -- 2nd Normal Form (2NF),
5. -- 3rd Normal Form (3NF).

1.3.8 Modelling problems

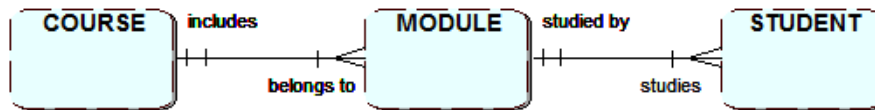
Care needs to be taken when modelling systems to ensure that you avoid producing a design which, if implemented, will not allow the system to extract all of the required information needed to answer user queries. With this in mind you should check your model for the following problems:

Fan trap

A fan trap is caused when it is not possible to link from one entity to another entity via a linking entity because the two 1:M relationships point away (fan out) from the linking entity. For example, suppose you want to know if a student is studying the module Database. The model below will not allow you to answer this query. Although the module is related to a course by the foreign key CourseID, there is no suitable link from course to student as there is no foreign key StudentID in course for student.

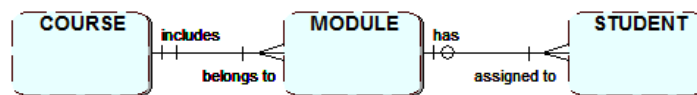


To resolve this problem a new relationship could be added, linking module directly with student, though a neater solution would be to rearrange the model as follows:-

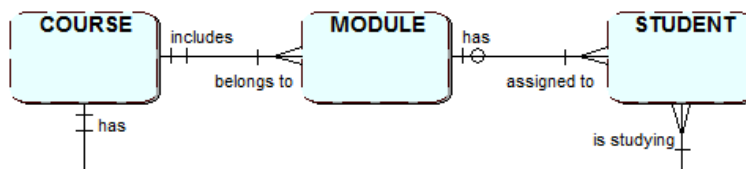


Chasm trap

A chasm trap is created when relationships between entity types indicate a route linking them, but due to optionality it is not possible to make the required connection for all occurrences. For example, suppose you want to identify which course a student is taking. The following model will not work if the student has not been assigned to a module. Although there is a link between course and module due to the Course_ID foreign key in the module relation, there would be no link between student and module if the student was not taking a module – there would be no foreign key Module_ID in student to provide a link to the module relation.



This problem can be resolved by adding a 1:M relationship linking course to student directly.



Finally, the designer uses the resulting system-specific database schema in the subsequent physical-design phase, in which the physical features of the database are specified. The physical schema of a database can be changed relatively easily after an application has been built. However, changes to the logical schema are usually harder to carry out, since they may affect a number of queries and updates scattered across application code. It is therefore important to carry out the database design phase with care, before building the rest of the database application.

The implementation phase is where you install the DBMS on the required hardware, optimize the database to run best on that hardware and software platform, and create the database and load the data. The initial data could be either new data captured directly or existing data imported from a MariaDB database or another DBMS. You also establish database security in this phase and give the various users that you've identified access applicable to their requirements. Finally, you also initiate backup plans in this phase.

The following are steps in the implementation phase:

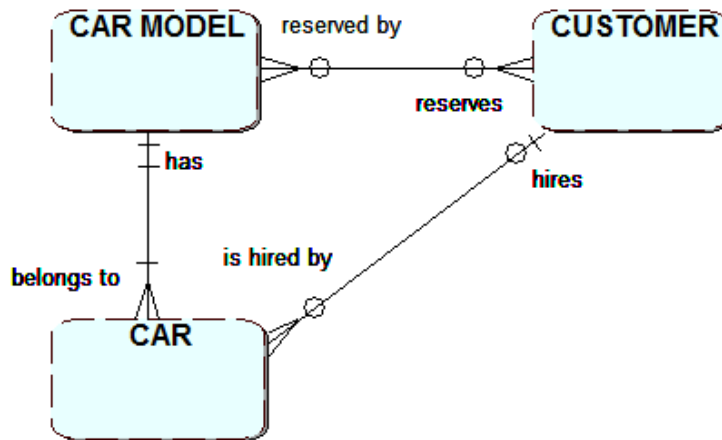
1. Install the DBMS.
2. Tune the setup variables according to the hardware, software and usage conditions.
3. Create the database and tables.
4. Load the data.
1. Set up the users and security.
2. Implement the backup regime.

1.1.8 Conclusion

A database management system provides us with mechanisms for storing and organizing data to facilitate easy access to and manipulation of the data. Today's most popular database systems are relational databases, although enhanced models exist.

1.1.9 Tutor-Marked assignment

1. Produce a logical design for the ERD



3. State reasons why normalization is needed in a database
4. Describe with suitable illustration, the various forms of normalization
5. Describe the various stages of database design

1.1.10. References and Further Readings

- Atzeni, P., Ceri, S., Paraboschi, S., & Torlone, R. (1999). *Database systems: concepts, languages & architectures* (Vol. 1). London: McGraw-Hill.
- Batini, C., Ceri, S., and Navathe, S. (1992) Database Design: An Entity – Relationship Approach,
- Bernstein, P. (1976), "Synthesizing Third Normal Form Relations from Functional Dependencies", TODS, 1:4, December 1976.
- Codd, E (1970) "A Relational Model for Large Shared Data BANKs" CACM, 136, June 1970.
- David M. Kroenke, David J. Auer (2008). Database Concepts. New Jersey. Prentice Hall
- Elmasri Navathe (2003). Fundamentals of Database Systems. England. Addison Wesley.
- Fred R. McFadden, Jeffrey A. Hoffer (1994). Modern Database management. England. Addison Wesley Longman
- Graeme C. Simsion, Graham C. Witt (2004). Data Modeling Essentials. San Francisco. Morgan Kaufmann
- Maier, D. (1983). The Theory of Relational Databases
- Pratt Adamski, Philip J. Pratt (2007). Concepts of Database Management. United States. Course Technology.
- Singh, S. K. (2011). *Database systems: Concepts, design and applications*. Pearson Education India.